

Một số nghiên cứu về lời giải cho SPOJ375 QTREE

Yang Zhe

Ngày 3 tháng 1 năm 2007

Abstract

Tuy bài này đã được các tác giả đi trước nghiên cứu [3] và đã có một lời giải khá thỏa đáng với độ phức tạp $O(n \log n + q\sqrt{n} \log n)$, vẫn còn một số thuật toán tốt chưa được nhắc đến. Bài viết này bắt đầu từ mô hình tổng quát của dạng bài này, tức bài toán cây động, rồi giới thiệu ngắn gọn cách cải biến mô hình đó để áp dụng cho QTREE.

Trước hết bài viết trình bày lời giải dùng trực tiếp **Link-Cut Trees** với độ phức tạp thời gian $O((n + q) \log n)$. Vì lời giải này không khai thác đầy đủ đặc điểm riêng của bài nên hiệu quả thực tế không tốt. Sau đó là một lời giải khai thác tính chất của đề bài, có độ phức tạp $O(n + q \log^2 n)$ và chạy khá tốt trong thực tế; trước khi tác giả nộp lời giải cuối cùng, chương trình này xếp thứ hai trên SPOJ.

Kết quả đã có không ngăn được việc tiếp tục tìm kiếm. Khi thay cấu trúc dữ liệu duy trì đường đi trong lời giải đó bằng **Splay Tree**, độ phức tạp giảm thành $O((n + q) \log n)$, nhưng do hằng số của Splay Tree rất lớn nên hiệu quả thực tế vẫn chưa lý tưởng. Sau khi so sánh và phân tích các lời giải trên, bài viết đề xuất một cấu trúc dữ liệu tính gọi là “cây nhị phân cân bằng toàn cục”. Dùng cấu trúc này để duy trì cả cây cho ta lời giải $O((n + q) \log n)$, và hiệu quả thực tế cũng khá tốt, nhanh hơn một chút so với thuật toán $O(n + q \log^2 n)$ đã nói ở trên.

Hy vọng bài viết có thể gợi mở thêm cho việc nghiên cứu các bài toán loại này.

Contents

1	Mô tả bài toán và phân tích sơ bộ	2
2	Bài toán cây động	2
3	Link-Cut Trees	3
3.1	Định nghĩa Link-Cut Trees	3
3.2	Các thao tác của Link-Cut Trees	3
3.2.1	Access	3
3.2.2	Find Root	4
3.2.3	Cut	4
3.2.4	Join	4
3.2.5	Mã giả của các thao tác	4
3.3	Phân tích độ phức tạp của Link-Cut Trees	5
3.3.1	Phân rã đường đi nặng-nhẹ	5
3.3.2	Chứng minh số lần đổi Preferred Child là $O(\log n)$ khấu hao	6
3.3.3	Chứng minh tổng thời gian Splay là $O(\log n)$ khấu hao	6

4	Lời giải cho bài toán	7
4.1	Tổng quan các lời giải	7
4.2	Lời giải cây động	8
4.2.1	Một mẹo cài đặt Splay Tree	8
4.3	Phương pháp phân rã đường đi nặng-nhẹ	10
4.4	Dùng cây đoạn hoặc cây nhị phân ảo để duy trì đường đi	10
4.5	Dùng Splay Tree để duy trì đường đi	10
4.6	Tìm kiếm một cấu trúc dữ liệu duy trì đường đi tốt hơn	10
4.7	Lời giải dùng cây nhị phân cân bằng toàn cục	11
4.7.1	Cây ảo	11
4.7.2	Định nghĩa và xây dựng cây nhị phân cân bằng toàn cục	11
4.7.3	Lời giải dùng cây nhị phân cân bằng toàn cục	13
4.7.4	Cây nhị phân “cân bằng toàn cục”	14
4.8	So sánh và tổng kết	14

A	Lời cảm ơn	15
----------	-------------------	-----------

1 Mô tả bài toán và phân tích sơ bộ

Cho một cây có n đỉnh, $n \leq 10000$. Mỗi cạnh có một trọng số. Cần duy trì một cấu trúc dữ liệu hỗ trợ các thao tác sau:

1. sửa trọng số của một cạnh;
2. hỏi trọng số cạnh lớn nhất trên đường đi duy nhất giữa hai đỉnh.

Tổng số thao tác là q .

Với thao tác thứ hai, chỉ cần tìm tổ tiên chung gần nhất của hai đỉnh được hỏi, rồi lần lượt tìm trọng số cạnh lớn nhất trên hai đường đi từ hai đỉnh đó lên tổ tiên chung. Hiển nhiên, tính trực tiếp sẽ quá chậm. Không khó nhận ra bài toán yêu cầu một cấu trúc dữ liệu duy trì thông tin trọng số trên đường đi từ một đỉnh lên gốc. Tuy nhiên, như vậy vẫn chưa đủ để giải quyết bài toán. Ta hãy xét trước một bài toán tổng quát hơn: bài toán cây động.

2 Bài toán cây động

Bài toán cây động yêu cầu duy trì một rừng gồm nhiều cây có gốc, trong đó các con của mỗi đỉnh không có thứ tự. Cấu trúc dữ liệu cần hỗ trợ tách cây, nối cây, một số thao tác trên đường đi từ một đỉnh lên gốc của nó, và một số thao tác trên cây con của một đỉnh. Nội dung đầy đủ có thể xem trong [1].

Ở đây ta chỉ xét một phiên bản đơn giản hơn, chỉ gồm các thao tác thay đổi hình dạng cây và các thao tác trên đường đi từ một đỉnh lên gốc. Nếu có thao tác trên cây con thì cần dùng một cấu trúc khác gọi là **Euler-Tour Trees**, nằm ngoài phạm vi bài viết này. Cụ thể, ta duy trì một cấu trúc dữ liệu hỗ trợ:

- **MAKE-TREE()** – tạo một cây mới chỉ gồm một đỉnh;
- **CUT(v)** – xóa cạnh giữa v và cha của nó $\text{parent}(v)$, tương đương tách cây con gốc v ra;

- $\text{JOIN}(v, w)$ – cho v trở thành con mới của w , trong đó v là gốc của một cây và v, w nằm trong hai cây khác nhau;
- $\text{FIND-ROOT}(v)$ – trả về gốc của cây chứa v .

Khi hiểu rõ bài toán này, ta cũng dễ mở rộng cấu trúc dữ liệu để duy trì một số thông tin trên đường đi từ mỗi đỉnh lên gốc của cây chứa nó, chẳng hạn tổng trọng số, trọng số cạnh lớn nhất, hoặc độ dài đường đi.

3 Link-Cut Trees

Link-Cut Trees là cấu trúc dữ liệu do Sleator và Tarjan phát minh để giải bài toán cây động [1]. Cấu trúc này thực hiện mỗi thao tác ở trên trong thời gian khấu hao $O(\log n)$.

Phần này trước hết giới thiệu khái niệm Link-Cut Trees, sau đó là các thao tác mà cấu trúc hỗ trợ, cuối cùng phân tích độ phức tạp.

3.1 Định nghĩa Link-Cut Trees

Ta nói một đỉnh được truy cập nếu vừa thực hiện thao tác **ACCESS** trên đỉnh đó.

Nếu trong cây con của đỉnh v , đỉnh được truy cập sau cùng nằm trong cây con gốc w , với w là con của v , thì gọi w là **Preferred Child** của v . Nếu đỉnh được truy cập sau cùng chính là v , thì v không có Preferred Child. Cạnh nối một đỉnh với Preferred Child của nó được gọi là **Preferred Edge**. Những đường đi tối đại được nối bởi Preferred Edge được gọi là **Preferred Path**.

Như vậy, cả cây được chia thành một số Preferred Path. Mỗi Preferred Path được duy trì bằng một cây cân bằng, dùng độ sâu của đỉnh trên đường đi làm khóa: trong cây cân bằng đó, mọi đỉnh ở cây con trái của một đỉnh đều nằm phía trên đỉnh ấy trên Preferred Path, còn mọi đỉnh ở cây con phải đều nằm phía dưới. Cây cân bằng này phải hỗ trợ tách và hợp nhất; ở đây ta chọn Splay Tree. Ta gọi cây cân bằng đó là một **Auxiliary Tree**.

Khi đã biết các Preferred Path mà cây T được tách thành, chỉ cần biết thêm quan hệ nối giữa các đường đi đó là biểu diễn được T . Ta dùng **Path Parent** để ghi cha của đỉnh cao nhất trên Preferred Path tương ứng với mỗi Auxiliary Tree. Nếu đỉnh cao nhất của Preferred Path chính là gốc, thì Path Parent của Auxiliary Tree đó là null.

Link-Cut Trees biểu diễn mỗi cây T trong rừng cần duy trì bằng nhiều Auxiliary Tree, rồi nối các Auxiliary Tree đó bằng Path Parent.

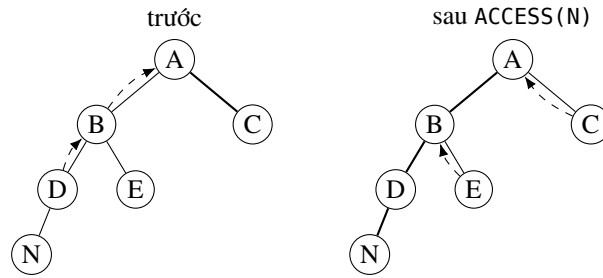
3.2 Các thao tác của Link-Cut Trees

3.2.1 Access

ACCESS là nền tảng của mọi thao tác trong Link-Cut Trees. Giả sử gọi $\text{ACCESS}(v)$. Khi đó đường đi từ v lên gốc trở thành một Preferred Path mới. Nếu trên đường đi có một đỉnh u không phải Preferred Child của cha $\text{parent}(u)$, thì vì Preferred Child của $\text{parent}(u)$ sẽ đổi thành u , Preferred Path vốn chứa $\text{parent}(u)$ sẽ không còn chứa phần từ $\text{parent}(u)$ trở lên.

Sơ đồ sau tóm tắt tác dụng của thao tác này. Các cạnh đậm là Preferred Path; sau $\text{ACCESS}(N)$, đường từ N lên gốc trở thành đường ưu tiên mới, còn các đoạn bị tách ra được giữ bằng Path Parent giữa các Auxiliary Tree.

Cách thực hiện **ACCESS** có vẻ bất ngờ: ta cập nhật trực tiếp các Auxiliary Tree cần thay đổi. Thoạt nhìn cách này không hiệu quả, nhưng phần sau sẽ chứng minh độ phức tạp khấu hao của nó là $O(\log n)$.



Hình 1: Minh họa ACCESS: đường truy cập trở thành Preferred Path, các đường cũ được nối lại bằng Path Parent.

Trước hết, vì đã truy cập v , Preferred Child của v phải biến mất. Ta xoay v lên gốc của Auxiliary Tree chứa nó. Nếu v có con phải trong Auxiliary Tree đó, tức Preferred Child cũ của v , thì tách cây con phải của v ra khỏi Auxiliary Tree hiện tại, và đặt Path Parent của Auxiliary Tree mới này là v .

Sau đó, nếu Preferred Path chứa v chưa chứa gốc của cây thật, gọi Path Parent của nó là u . Ta xoay u lên gốc của Auxiliary Tree chứa u , thay cây con phải của u trong Auxiliary Tree đó bằng Auxiliary Tree chứa v , rồi đặt Path Parent của cây con phải cũ của u thành u . Lặp quá trình này cho đến khi gặp Preferred Path chứa gốc.

3.2.2 Find Root

Sau ACCESS(v), gốc chắc chắn là đỉnh nhỏ nhất trong Auxiliary Tree chứa v . Trước hết xoay v lên gốc của Auxiliary Tree đó. Sau đó bắt đầu từ v đi liên tục sang trái trong Auxiliary Tree cho đến khi không thể đi tiếp; đỉnh đó chính là gốc cần tìm. Vì Auxiliary Tree được lưu bằng Splay Tree, ta còn thực hiện Splay trên đỉnh gốc vừa tìm được.

3.2.3 Cut

Trước hết truy cập v , rồi xoay v lên gốc của Auxiliary Tree chứa nó. Sau đó ngắt liên kết giữa v và cây con trái của nó trong Auxiliary Tree.

3.2.4 Join

Trước hết truy cập v , rồi sửa Path Parent của Auxiliary Tree chứa v thành w , sau đó truy cập lại v .

3.2.5 Mã giả của các thao tác

```

procedure Access( $v$ )
   $u \leftarrow v$ 
   $v \leftarrow \text{null}$ 
  repeat
    Splay( $u$ )
    path-parent[right-child[ $u$ ]]  $\leftarrow u$ 
    right-child[ $u$ ]  $\leftarrow v$ 
    path-parent[ $v$ ]  $\leftarrow \text{null}$ 
     $v \leftarrow u$ 
     $u \leftarrow \text{path-parent}[u]$ 
  until  $u = \text{null}$ 
end procedure

```

```

function FindRoot(v)
    Access(v)
    Splay(v)
    while left-child[v] != null do
        v <- left-child[v]
    end while
    Splay(v)
    return v
end function

procedure Cut(v)
    Access(v)
    left-child[v] <- null
end procedure

procedure Join(v, w)
    Access(v)
    path-parent[v] <- w
    Access(v)
end procedure

```

3.3 Phân tích độ phức tạp của Link-Cut Trees

Từ mã giả có thể thấy, ngoài ACCESS, mọi thao tác khác có độ phức tạp khấu hao nhiều nhất $O(\log n)$. Vì vậy chỉ cần phân tích độ phức tạp của ACCESS.

Trong phân tích thao tác ACCESS, ta chia thời gian thành hai phần. Thứ nhất, chứng minh số lần đổi Preferred Child là $O(\log n)$ khấu hao. Thứ hai, chứng minh tổng thời gian của mọi thao tác Splay trong một lần ACCESS là $O(\log n)$ khấu hao.

3.3.1 Phân rã đường đi nặng-nhẹ

Phân rã đường đi nặng-nhẹ là một kỹ thuật phân tích áp dụng cho mọi cây. Nó chia các cạnh của cây thành hai loại: cạnh nặng và cạnh nhẹ.

Ký hiệu $\text{size}(i)$ là số đỉnh trong cây con gốc i . Gọi u là đứa con có $\text{size}(u)$ lớn nhất trong các con của v ; nếu có nhiều đứa con như vậy thì chọn tùy ý. Cạnh (v, u) được gọi là cạnh nặng, còn các cạnh (v, w) từ v đến những đứa con khác w được gọi là cạnh nhẹ, ngay cả khi $\text{size}(w) = \text{size}(u)$. Mỗi đỉnh có con thì có duy nhất một cạnh nặng đi ra từ nó. Từ định nghĩa này suy ra ngay:

Tính chất 3.1. Nếu u là con của v và (v, u) là một cạnh nhẹ, thì

$$\text{size}(u) < \text{size}(v)/2.$$

Chứng minh. Giả sử $\text{size}(u) \geq \text{size}(v)/2$. Vì (v, u) là cạnh nhẹ, tồn tại một cạnh nặng (v, w) đi ra từ v . Theo định nghĩa cạnh nặng,

$$\text{size}(v) \geq 1 + \text{size}(w) + \text{size}(u) \geq 2 \text{size}(u) + 1 \geq 1 + \text{size}(v),$$

mâu thuẫn.

Gọi $\text{light-depth}(v)$ là số cạnh nhẹ trên đường từ v lên gốc.

Bổ đề 3.1.

$$\text{light-depth}(v) \leq \lg n.$$

Chứng minh. Theo Tính chất 3.1, mỗi khi đi qua một cạnh nhẹ thì kích thước cây con hiện tại giảm xuống nhiều nhất còn một nửa. Ban đầu kích thước cây con không quá n , còn cây con của v có ít nhất một đỉnh, nên đi qua nhiều nhất $\lg n$ cạnh nhẹ.

3.3.2 Chứng minh số lần đổi Preferred Child là $O(\log n)$ khấu hao

Để đếm số lần Preferred Child thay đổi, trước hết ta thực hiện phân rã nặng-nhẹ trên cây T .

Ngoài việc Preferred Child của v biến mất, mỗi lần Preferred Child thay đổi đều sinh ra một Preferred Edge mới. Vì vậy chỉ cần đếm số Preferred Edge mới được sinh ra.

Theo Bổ đề 3.1, trong quá trình ACCESS có nhiều nhất $\lg n$ Preferred Edge nhẹ mới. Tuy nhiên, một lần truy cập có thể sinh ra nhiều Preferred Edge nặng. Ta đếm chúng như sau. Số lần một cạnh nặng trở thành Preferred Edge nhiều nhất bằng số lần nó từ Preferred Edge trở lại cạnh thường, cộng với tổng số cạnh, vì cuối cùng có thể còn lại toàn bộ các cạnh nặng là Preferred Edge. Mỗi lần một cạnh nặng từ Preferred Edge trở lại cạnh thường thì tương ứng có một cạnh nhẹ trở thành Preferred Edge. Do đó tổng số lần cạnh nặng trở thành Preferred Edge nhiều nhất bằng tổng số lần cạnh nhẹ trở thành Preferred Edge cộng với tổng số cạnh. Trung bình trên mỗi thao tác ACCESS, số cạnh nặng trở thành Preferred Edge là $O(\log n)$.

Tóm lại, số lần Preferred Child thay đổi là $O(\log n)$ khấu hao.

3.3.3 Chứng minh tổng thời gian Splay là $O(\log n)$ khấu hao

Phân tích độ phức tạp của Splay thường dùng phương pháp thế năng.

Gán cho mỗi đỉnh u trong Splay Tree một trọng số dương $w(u)$, và gọi $s(u)$ là tổng trọng số các đỉnh trong cây con của u . Định nghĩa hàm thế năng

$$\Phi = \sum_u \lg s(u).$$

Các kết quả chuẩn sau đây được dùng; trong đó $s(u)$ là giá trị trước thao tác và $s'(u)$ là giá trị sau thao tác.

Bổ đề 3.2. Trong thao tác SPLAY(v), chi phí khấu hao của bước zig-zig hoặc zag-zag thỏa

$$\widehat{\text{cost}} \leq 3(\lg s'(v) - \lg s(v)).$$

Hằng số 3 ở đây không thể cải thiện.

Bổ đề 3.3. Trong thao tác SPLAY(v), chi phí khấu hao của bước zig-zag hoặc zag-zig thỏa

$$\widehat{\text{cost}} \leq 2(\lg s'(v) - \lg s(v)) \leq 3(\lg s'(v) - \lg s(v)).$$

Bổ đề 3.4. Trong thao tác SPLAY(v), chi phí khấu hao của bước zig hoặc zag thỏa

$$\widehat{\text{cost}} \leq \lg s'(v) - \lg s(v) + 1 \leq 3(\lg s'(v) - \lg s(v)) + 1.$$

Chứng minh ba bổ đề này khá đơn giản; độc giả quan tâm có thể tự chứng minh hoặc tham khảo tài liệu liên quan. Từ đó suy ra:

Định lý 3.5 (Access Theorem). Chi phí khấu hao của $\text{SPLAY}(v)$ thỏa

$$\widehat{\text{cost}} \leq 3(\lg s'(v) - \lg s(v)) + 1.$$

Hằng số 3 ở đây không thể cải thiện.

Trong bài toán hiện tại, vì một thao tác ACCESS thực hiện nhiều thao tác SPLAY (khấu hao $O(\log n)$ lần), ta cần chọn một hàm trọng số phù hợp, thay vì đơn giản đặt $w(u) = 1$. Nếu xem các cạnh trong Auxiliary Tree là cạnh thật, còn Path Parent là cạnh ảo, thì tổng số đỉnh của một cây con bằng 1 cộng với tổng số đỉnh của mọi cây con của nó, kể cả các cây con nối bằng cạnh ảo. Đặt

$$w(u) = 1 + \text{tổng số đỉnh trong các cây con có Path Parent là } u,$$

khi đó $s(u)$ chính là số đỉnh trong cây con của u .

Giả sử trong quá trình $\text{ACCESS}(v)$, ta lần lượt thực hiện Splay trên $v_0 = v, v_1 = \text{path-parent}[v_0], \dots, v_k = \text{path-parent}[v_{k-1}]$. Khi đó chi phí khấu hao của thao tác ACCESS là

$$\begin{aligned} \widehat{\text{cost}} &\leq \sum_{i=0}^k 3(\lg s'(v_i) - \lg s(v_i)) + 1 \\ &\leq 3 \left[\sum_{i=1}^k (\lg s(v_i) - \lg s(v_{i-1})) + \lg s(v_0) - \lg s(v_k) \right] + k \\ &= 3(\lg s(v_0) - \lg s(v_k)) + k \\ &\leq 3 \lg n + \text{số lần Preferred Child thay đổi}. \end{aligned}$$

Dòng giữa trong bản in không rõ một vài chỉ số; công thức trên giữ đúng ý nghĩa telescoping của chứng minh trong bài gốc.

Vì đã chứng minh số lần Preferred Child thay đổi là $O(\log n)$ khấu hao, thao tác ACCESS cũng có độ phức tạp khấu hao $O(\log n)$.

4 Lời giải cho bài toán

4.1 Tổng quan các lời giải

Với các kiến thức trên, ta có thể giải tốt bài toán. Bài viết thảo luận bốn lời giải, như Bảng 1.

Số	Tóm tắt lời giải	Độ phức tạp
1	Cây động, dùng Link-Cut Trees	$O((n + q) \log n)$
2	Phân rã nặng-nhẹ, dùng cây đoạn hoặc cây nhị phân ảo cho từng đường	$O(n + q \log^2 n)$
3	Phân rã nặng-nhẹ, dùng Splay Tree cho từng đường	$O((n + q) \log n)$
4	Phân rã nặng-nhẹ, dùng một cây nhị phân cân bằng toàn cục cho mọi đường	$O((n + q) \log n)$

Table 1: Các lời giải được trình bày trong bài

Các lời giải trong bài gốc đều có chương trình tham khảo. Thời gian chạy của chúng được ghi trong Bảng 2.

Cần chú ý rằng lời giải 2 và 3 tuy chỉ khác cấu trúc dữ liệu dùng để duy trì đường đi, nhưng vì độ phức tạp thời gian khác nhau, và việc so sánh chúng đóng vai trò quan trọng trong việc dẫn tới lời giải 4, nên được liệt kê riêng. Hai chương trình của lời giải 2 chỉ khác cách cài đặt; thời gian chạy gần nhau và không đủ để đánh giá cây đoạn hay cây nhị phân ảo tốt hơn.

Lời giải	Chương trình tham khảo	Thời gian
1	QTREEdynamic-tree link-cut-trees.pas	4.58 giây
2, cây đoạn	QTREEheavy-light-decomposition segment-tree.pas	2.65 giây
2, cây nhị phân ảo	QTREEheavy-light-decomposition imaginary-bst.pas	2.37 giây
3	QTREEheavy-light-decomposition splay-tree.pas	4.28 giây
4	QTREEheavy-light-decomposition global-balanced-bst.pas	2.06 giây

Table 2: So sánh thời gian chạy của các chương trình

4.2 Lời giải cây động

Dùng Link-Cut Trees để duy trì cây. Ta chỉ cần mở rộng Auxiliary Tree đã nói ở trên: trong Auxiliary Tree, với mỗi đỉnh ghi trọng số cạnh nối nó với cha là cost , và ghi maxcost , tức giá trị cost lớn nhất trong cây con gốc đỉnh đó. Vì phép lấy lớn nhất có tính kết hợp, maxcost có thể được duy trì trong $O(1)$ khi xoay Auxiliary Tree.

Với thao tác sửa trọng số cạnh, chỉ cần cập nhật cost của đỉnh tương ứng trong Auxiliary Tree chứa nó, rồi Splay đỉnh đó để duy trì các giá trị maxcost liên quan trong Auxiliary Tree. Rõ ràng thao tác này mất $O(\log n)$.

Với truy vấn trọng số cạnh lớn nhất trên đường đi từ u đến v , trước hết $\text{ACCESS}(u)$. Sau đó, trong quá trình $\text{ACCESS}(v)$, khi đi tới Auxiliary Tree chứa gốc (tức chứa u), đỉnh đang xét đúng là tổ tiên chung gần nhất của u và v ; gọi đỉnh đó là d . Khi đó maxcost của cây con phải của d trong Auxiliary Tree chứa d là trọng số lớn nhất trên đường từ u đến d , còn maxcost của Auxiliary Tree chứa v là trọng số lớn nhất trên đường từ v đến d . Đáp án là giá trị lớn hơn trong hai maxcost đó. Vì Access có độ phức tạp khâu hao $O(\log n)$, truy vấn cũng mất $O(\log n)$.

Do thế năng ban đầu của Link-Cut Trees là $O(n \log n)$, tổng độ phức tạp của phương pháp này là $O((n + q) \log n)$. Tuy nhiên, vì dùng Splay Tree làm cấu trúc cho Auxiliary Tree, mà hằng số của Splay Tree rất lớn, thuật toán này không dễ vượt giới hạn thời gian 5 giây. Kể cả tác giả của [3] cũng không dùng cách này để qua toàn bộ dữ liệu. Phần tiếp theo mô tả một mẹo cài đặt Splay Tree trong chương trình của tác giả.

4.2.1 Một mẹo cài đặt Splay Tree

Trong Định lý 3.5, ta thấy hằng số của Splay Tree bằng ba lần thời gian của một phép xoay đơn. Trong một phép xoay đơn, không chỉ phải sửa hình dạng cục bộ của Splay Tree mà còn phải duy trì thuộc tính maxcost ; viết tốt phần này rất quan trọng với hiệu quả thực tế của Splay Tree. Đồng thời, cách viết Splay Tree thông thường phải kiểm tra đỉnh có phải gốc không, hoặc là con nào của cha, cũng tốn nhiều thời gian. Vì vậy tác giả tối ưu các thao tác này và thu được cách viết SPLAY, ROTATE, UPDATE vừa ngắn gọn vừa hiệu quả.

Giải thích một số biến: để tiện cài đặt UPDATE, đặt một đỉnh rỗng giả null với $\text{maxcost} = -\text{infinity}$. Để tiện cài đặt ROTATE, giả sử con trái của đỉnh x lưu ở $\text{child}[x][0]$, con phải lưu ở $\text{child}[x][1]$. Nếu x không phải gốc, $\text{parent}[x]$ lưu cha của x , còn $\text{nodetype}[x]$ ghi x là con nào của cha, tức $\text{child}[\text{parent}[x]][\text{nodetype}[x]] = x$. Nếu x là gốc, $\text{parent}[x]$ lưu Path Parent của Auxiliary Tree này và $\text{nodetype}[x] = 2$.

```

procedure Update(node)           // cập nhật maxcost của node
  k <- cost[node]
  x <- maxcost[child[node][0]]
  y <- maxcost[child[node][1]]

```



```

    if x < y then
        x <- y
    end if
    if k < x then
        k <- x
    end if
    maxcost[node] <- k
end procedure

procedure Rotate(node, x)    // xoay node co nodetype x len vi tri cua cha
    p <- parent[node]
    y <- nodetype[p]

    tmp <- parent[p]
    parent[node] <- tmp
    nodetype[node] <- y
    if y != 2 then
        child[tmp][y] <- node
    end if

    y <- 1 - x
    tmp <- child[node][y]
    child[p][x] <- tmp
    parent[tmp] <- p
    nodetype[tmp] <- x

    parent[p] <- node
    nodetype[p] <- y
    child[node][y] <- p

    Update(p)
end procedure

procedure Splay(node)
    repeat
        a <- nodetype[node]
        if a = 2 then
            break
        end if
        p <- parent[node]
        b <- nodetype[p]

        if a = b then
            Rotate(p, a)    // buoc dau cua zig-zig hoac zag-zag
        else
            Rotate(node, a)    // buoc dau cua zig-zag, zag-zig, zig hoac zag
        end if
        if b = 2 then
            break    // p la goc cu, node da duoc xoay len goc
        end if
        Rotate(node, b)    // buoc thu hai
    until false
    Update(node)
end procedure

```

Trong quá trình SPLAY(node), cách viết thông thường sẽ gọi UPDATE trên một số đỉnh quá nhiều lần, nhưng thực ra không cần thiết. Chỉ cần trong ROTATE cập nhật cha của đỉnh được xoay, rồi cuối SPLAY cập nhật một lần trên node. Vì UPDATE là thao tác tốn thời gian, tối ưu này loại bỏ khoảng một nửa số lần UPDATE thừa và cải thiện đáng kể hiệu năng chương trình.

4.3 Phương pháp phân rã đường đi nặng-nhẹ

Trong bài này, hình dạng cây không thay đổi, nên ta hoàn toàn không dùng tới các thao tác thay đổi hình dạng cây trong bài toán cây động. Với một bài toán đặc biệt hơn như vậy, đáng lẽ phải có một thuật toán không phức tạp hơn Link-Cut Trees và không chậm hơn $O((n + q) \log n)$.

Link-Cut Trees dùng các khái niệm như Preferred Child để cho phép hình dạng cây thay đổi. Liệu ta có thể bỏ các khái niệm đó không? Sau khi phân tích, ta thấy có thể. Chỉ cần áp dụng kỹ thuật phân rã đường đi nặng-nhẹ trong phần phân tích độ phức tạp của Link-Cut Trees là thu được lời giải nhanh cho bài này.

4.4 Dùng cây đoạn hoặc cây nhị phân ảo để duy trì đường đi

Trước hết, thực hiện phân rã nặng-nhẹ trên cây. Gọi một đường đi tối đại gồm các cạnh nặng là **đường nặng**. Khi đó cây được chia thành nhiều đường nặng. Ta duy trì mỗi đường nặng bằng một cây đoạn hoặc một cây nhị phân ảo. Ta mượn tên Auxiliary Tree để gọi mỗi cây đoạn hoặc cây nhị phân ảo này.

Với thao tác sửa trọng số cạnh, chỉ cần sửa cost của đỉnh tương ứng và duy trì Auxiliary Tree chứa nó.

Với truy vấn trọng số cạnh lớn nhất trên đường giữa u và v , ta dùng cách ngây thơ để tìm tổ tiên chung gần nhất của hai Auxiliary Tree tương ứng với u và v . Theo Bổ đề 3.1, từ bất kỳ đỉnh nào lên gốc đều đi qua nhiều nhất $\lg n$ đường nặng, nên nhiều nhất $O(\log n)$ Auxiliary Tree. Trong quá trình tìm tổ tiên chung gần nhất, trên mỗi Auxiliary Tree đi qua, ta có thể lấy trọng số cạnh lớn nhất trên phần đường đi thuộc Auxiliary Tree đó trong $O(\log n)$. Cuối cùng tính thêm phần đường đi nằm trong Auxiliary Tree chứa tổ tiên chung gần nhất. Giá trị lớn nhất trong tất cả các giá trị này là đáp án.

Vì sửa trọng số mất $O(\log n)$, còn trả lời truy vấn mất $O(\log^2 n)$, tổng độ phức tạp là $O(n + q \log^2 n)$.

4.5 Dùng Splay Tree để duy trì đường đi

Trong lời giải này, chỉ cần thay cấu trúc dữ liệu duy trì đường đi ở lời giải trước bằng Splay Tree. Dùng cùng hàm thế năng như trong phân tích độ phức tạp của cây động, có thể chứng minh thời gian khấu hao trả lời truy vấn là $O(\log n)$. Vì thế năng ban đầu là $O(n \log n)$, tổng độ phức tạp là $O((n + q) \log n)$.

Nhưng trong Bảng 2, ta thấy thời gian chạy của chương trình này không lý tưởng. Điều đó dẫn tới câu hỏi sau.

4.6 Tìm kiếm một cấu trúc dữ liệu duy trì đường đi tốt hơn

Có tồn tại một cấu trúc dữ liệu để duy trì đường đi, không cần mạnh như Splay Tree, nhưng đồng thời có độ phức tạp lý thuyết như lời giải 3 và hiệu quả thực tế tốt như lời giải 2 hay không?

Trong phần thảo luận trên, ta thấy dùng cây đoạn hoặc cây nhị phân ảo để duy trì đường đi thì mỗi Auxiliary Tree đều cân bằng tuyệt đối, nhưng độ phức tạp lý thuyết lại cao hơn dùng Splay Tree. Vì sao?

Từ phân tích độ phức tạp khi dùng Splay Tree duy trì Auxiliary Tree, có thể thấy Splay Tree không xem từng Auxiliary Tree một cách cô lập, mà thông qua Path Parent xem các Auxiliary Tree như một tổng thể. Điều này đem lại cân bằng toàn cục, nên mỗi truy vấn có độ phức tạp khấu hao $O(\log n)$.

Cây đoạn hoặc cây nhị phân ảo không tận dụng liên hệ giữa các Auxiliary Tree. Dù từng Auxiliary Tree cân bằng đến đâu, việc mất phương hướng toàn cục lại dẫn tới mất cân bằng trên tổng thể.

Vì vậy, nếu tận dụng Path Parent giữa các Auxiliary Tree và tùy chỉnh hình dạng phù hợp cho mỗi Auxiliary Tree nhằm đạt cân bằng toàn cục, ta có hy vọng thu được thuật toán đang tìm kiếm.

4.7 Lời giải dùng cây nhị phân cân bằng toàn cục

Theo suy nghĩ ở phần trước, ta xem tất cả Auxiliary Tree như một tổng thể và cần tìm cách chọn hình dạng phù hợp cho mỗi Auxiliary Tree. Vì phương pháp này đạt cân bằng trên tổng thể, ta gọi nó là **cây nhị phân cân bằng toàn cục**.

4.7.1 Cây ảo

Trước hết, ta xét từ góc nhìn toàn cục, chưa quan tâm hình dạng cụ thể của mỗi Auxiliary Tree mà chỉ xét quan hệ giữa chúng.

Ta gọi con của một đỉnh trong Auxiliary Tree là **con thật** của nó. Nếu gốc của một Auxiliary Tree là r và $\text{path-parent}[r] = u$, ta gọi r là một **con ảo** của u . Xét một cây có gốc là gốc của Auxiliary Tree cao nhất, được nối bởi cả cạnh thật và cạnh ảo; nhắc lại cách nhìn trong lời giải 2 và Link-Cut Trees, ta gọi cây này là **cây ảo**. Cây con của một đỉnh được định nghĩa là cây gồm chính nó và các cây con nối với nó bằng cạnh thật hoặc cạnh ảo; con trái và con phải là con của nó trong Auxiliary Tree chứa nó; con ảo là con nối với nó bằng cạnh ảo.

Bóng dáng của cây ảo xuất hiện trong mọi phương pháp đã thảo luận. Trong lời giải 1 và 3, ta duy trì một cây ảo động. Trong lời giải 2, ta duy trì một cây ảo tĩnh, nhưng độ sâu của nó là $O(\log^2 n)$.

Trong lời giải 4, mục tiêu là tìm một cây ảo có độ sâu $O(\log n)$. Tất nhiên có thể có nhiều cây như vậy; phần 4.7.4 sẽ thảo luận thêm. Ở đây, ta chỉ gọi những cây ảo thỏa định nghĩa cây nhị phân cân bằng toàn cục dưới đây là cây nhị phân cân bằng toàn cục.

4.7.2 Định nghĩa và xây dựng cây nhị phân cân bằng toàn cục

Làm thế nào để xây dựng một cây nhị phân cân bằng toàn cục? Trước hết giả sử ta đã dùng phân rã nặng-nhẹ để thu được một cây ảo, và đã xây dựng thành công một cây nhị phân cân bằng toàn cục. Ta nghiên cứu các tính chất của cây này.

Yêu cầu 4.1. Trong cây nhị phân cân bằng toàn cục, với mọi cây con, tổng số đỉnh trong cây con trái và cây con phải của nó đều không vượt quá một nửa tổng số đỉnh của cây con đó.

Yêu cầu này thực ra rất dễ thỏa. Giả sử các đỉnh trong Auxiliary Tree chứa gốc của cây con hiện tại, sau khi sắp theo khóa tăng dần, là $v_1, v_2, \dots, v_k$, trong đó k là số đỉnh của Auxiliary Tree đó. Đặt

$$w(u) = 1 + \text{tổng số đỉnh trong mọi cây con ảo có cha là } u,$$

và

$$s_i = \sum_{j=1}^i w(v_j).$$

Khi đó s_k là tổng số đỉnh của cây con này. Vì $s_0 = 0$ và $s_{k-1} < s_k$, tồn tại một số nguyên t sao cho

$$s_{t-1} < s_k/2 \leq s_t.$$

Chọn v_t làm gốc của Auxiliary Tree này. Khi đó tổng số đỉnh của cây con trái là $s_{t-1} < s_k/2$, còn tổng số đỉnh của cây con phải là $s_k - s_t \leq s_k/2$.

Tính chất 4.2. Trong cây nhị phân cân bằng toàn cục, với mọi cây con, tổng số đỉnh trong mỗi cây con ảo của nó nhỏ hơn một nửa tổng số đỉnh của cây con đó.

Chứng minh. Theo Tính chất 3.1, tổng số đỉnh của mọi cây con ảo nhỏ hơn một nửa hiệu giữa tổng số đỉnh của cây con hiện tại và tổng số đỉnh của cây con trái của nó, do đó cũng nhỏ hơn một nửa tổng số đỉnh của cây con hiện tại.

Yêu cầu 4.3 (cấu trúc con). Mọi cây con của một cây nhị phân cân bằng toàn cục cũng là cây nhị phân cân bằng toàn cục.

Định lý 4.1 (định lý cân bằng toàn cục). Độ sâu lớn nhất có thể của một cây nhị phân cân bằng toàn cục có n đỉnh, nếu gốc có độ sâu 0, thỏa

$$\text{max-depth}(n) \leq \lg n.$$

Chứng minh. Theo Yêu cầu 4.1, Tính chất 4.2 và Yêu cầu 4.3, số đỉnh trong cây con trái, cây con phải hoặc cây con ảo của gốc đều nhiều nhất bằng một nửa ban đầu, và mỗi cây con đó vẫn là một cây nhị phân cân bằng toàn cục. Do đó

$$\text{max-depth}(n) \leq \text{max-depth}(n/2) + 1 \leq \lg n.$$

Từ các nhận xét trên, ta có thể viết quy trình xây dựng cây nhị phân cân bằng toàn cục.

```
function Build(l, r)
    tìm nhị phân t sao cho
         $s[t-1] < (s[r] + s[l-1]) / 2 \leq s[t]$ 
    if  $t > l$  then
        left-child[t] <- Build(l, t - 1)
    end if
    if  $t < r$  then
        right-child[t] <- Build(t + 1, r)
    end if
    return t
end function

procedure Construct-Auxiliary-Tree(v[1..k])
    tính  $s[0..k]$ 
    Build(1, k)
end procedure

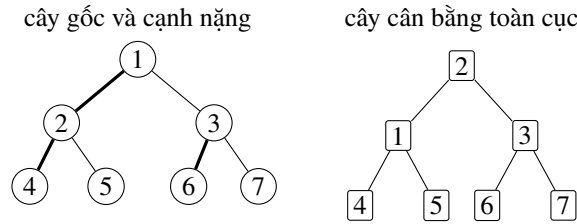
procedure Compute-Weight(node)
    size(node) <- 1
    for all canh (node, v) do
        Compute-Weight(v)
        size(node) <- size(node) + size(v)
    end for
    w(node) <- size(node)
    if tồn tại canh ngang (node, v) then
        w(node) <- w(node) - size(v)
    end if
end procedure
```

```

procedure Make-Global-Balanced-BST(tree)
  Heavy-Light-Decomposition(tree)
  Compute-Weight(root)
  for all đường nặng (v[1..k]),
    trong đó v[i] là cha của v[i+1] do
      Construct-Auxiliary-Tree(v[1..k])
  end for
end procedure

```

Từ mã giả có thể thấy quá trình xây dựng cây nhị phân cân bằng toàn cục mất $O(n \log n)$. Ý tưởng là trước hết gom từng chuỗi nặng thành một đoạn tuyến tính, rồi dựng cây nhị phân cân bằng trên các đoạn đó để độ sâu toàn cục vẫn bị chặn bởi $O(\log n)$.



Hình 2: Từ phân rã nặng-nhẹ sang cây nhị phân cân bằng toàn cục: các chuỗi nặng giữ liên hệ đường đi, còn cây cân bằng giới hạn độ sâu.

4.7.3 Lờ giải dùng cây nhị phân cân bằng toàn cục

Với thao tác sửa trọng số mà đề bài yêu cầu, ta chỉ cần duy trì Auxiliary Tree tương ứng. Quá trình này không nhắc lại nữa. Vì độ sâu của cây nhị phân cân bằng toàn cục là $O(\log n)$, thao tác này có độ phức tạp $O(\log n)$.

Với truy vấn trọng số cạnh lớn nhất trên đường đi, hãy nhớ lại cách làm trong lời giải 2. Ta chia truy vấn thành các phần trên những Auxiliary Tree “đi qua” và phần trên Auxiliary Tree chứa tổ tiên chung gần nhất. Phần truy vấn trong Auxiliary Tree chứa tổ tiên chung gần nhất có thể trả lời trong $O(\log n)$, nên nó không ảnh hưởng đến độ phức tạp của lời giải. Ta tập trung vào xử lý các Auxiliary Tree đi qua.

Trên mỗi Auxiliary Tree đi qua, ta chỉ cần biết giá trị cost lớn nhất từ đỉnh hiện tại, qua cha của nó, ..., đến đỉnh cao nhất của đường nặng. Trong Auxiliary Tree, điều này tương đương lấy cost lớn nhất trong các đỉnh có khóa không vượt quá khóa của đỉnh hiện tại. Rõ ràng, chỉ cần bắt đầu từ gốc của Auxiliary Tree đi xuống tới đỉnh hiện tại và dùng các giá trị maxcost đã duy trì là tính được giá trị cần tìm. Nếu độ sâu của gốc Auxiliary Tree là $\text{depth}(\text{root})$, còn độ sâu của đỉnh hiện tại là $\text{depth}(\text{node})$, truy vấn này tốn

$$O(\text{depth}(\text{root}) - \text{depth}(\text{node}) + 1).$$

Giả sử khi xử lý đỉnh truy vấn v_0 , các Auxiliary Tree đi qua lần lượt có gốc $r_0, r_1, \dots, r_k$, và Path Parent của r_i là v_{i+1} . Rõ ràng ta cần lần lượt tìm cost lớn nhất trên đoạn từ v_i đến r_i .

Theo kết luận trên, tổng thời gian trên các Auxiliary Tree đi qua là

$$\begin{aligned}
 O\left(\sum_{i=0}^k (\text{depth}(r_i) - \text{depth}(v_i) + 1)\right) &= O\left(\sum_{i=0}^k (\text{depth}(v_{i+1}) - \text{depth}(v_i))\right) \\
 &= O(\text{depth}(v_{k+1}) - \text{depth}(v_0)) \\
 &= O(\log n).
 \end{aligned}$$

Vì vậy, một truy vấn được trả lời trong $O(\log n)$.

Cũng có thể hiểu trực quan quá trình trả lời truy vấn như sau: ta tìm tổ tiên chung gần nhất của hai đỉnh trên cây nhị phân cân bằng toàn cục này (tổ tiên chung đó không nhất thiết là tổ tiên chung gần nhất của hai đỉnh trên cây gốc), và thu thập thông tin liên quan trên các đỉnh đi qua. Vì độ sâu của cây nhị phân cân bằng toàn cục là $O(\log n)$, tổng thời gian trả lời truy vấn là $O(\log n)$.

4.7.4 Cây nhị phân “cân bằng toàn cục”

Trong cây nhị phân cân bằng toàn cục, điều kiện then chốt ảnh hưởng đến độ sâu thực ra là Tính chất 4.2. Điều kiện này có thể nói lỏng thành: với mọi cây con, tổng số đỉnh trong mỗi cây con ảo nhỏ hơn c lần tổng số đỉnh của cây con đó, trong đó $c < 1$ là một hằng số. Ta gọi cây thỏa điều kiện này là một cây nhị phân “cân bằng toàn cục”. Rõ ràng Yêu cầu 4.1 và 4.3 vẫn có thể thỏa mãn. Tương tự Định lý 4.1, ta chứng minh được độ sâu của cây nhị phân “cân bằng toàn cục” không vượt quá

$$\log_{1/c} n = O(\log n),$$

chỉ là khi c lớn thì hằng số trong độ sâu cũng lớn.

Một cây nhị phân “cân bằng toàn cục” hiển nhiên cũng có thể dùng để giải bài này. Nhưng trên thực tế, ta có thể bảo đảm rằng hằng số c của cây nhị phân “cân bằng toàn cục” xây dựng được là ít nhất $1/2$. Vì vậy phương pháp trên không còn chỗ cải thiện. Ngay cả nếu tìm mọi cách để đạt một $c < 1/2$, cũng vô ích: khi đó yếu tố không chế độ sâu đã chuyển thành Yêu cầu 4.1, và xét trường hợp toàn bộ cây là một đường thẳng thì yêu cầu này không thể cải thiện nữa. Do đó, cây nhị phân cân bằng toàn cục đã trình bày ở trên là đủ tốt.

4.8 So sánh và tổng kết

Phân tích Bảng 2, ta thấy lời giải 3 và lời giải 2 tuy xuất phát từ cùng một hướng, nhưng do tính đặc biệt trong phân tích độ phức tạp của Splay Tree, lời giải 3 có độ phức tạp lý thuyết thấp hơn. Chỉ vì hằng số của Splay Tree khá lớn nên trong chạy thực tế, chương trình của lời giải 3 chậm hơn nhiều so với chương trình của lời giải 2.

Đồng thời, thời gian chạy của lời giải 1 và lời giải 3 rất gần nhau. Điều này cho thấy thao tác đổi Preferred Child trong Link-Cut Trees thực tế rất nhanh. Phần tốn thời gian nhất của Link-Cut Trees chính là thao tác dùng Splay Tree để duy trì đường đi. Điều thú vị là sau khi dùng Splay Tree làm cấu trúc dữ liệu duy trì đường đi, sự xuất hiện và ứng dụng của Link-Cut Trees trở nên rất tự nhiên: nó chỉ trả thêm một chi phí rất nhỏ nhưng nhận được khả năng mở rộng rất lớn. Khả năng mở rộng này chính là do cấu trúc dữ liệu đặc biệt Splay Tree và cách phân tích độ phức tạp đặc biệt của nó đem lại. Cùng với khả năng mở rộng lớn đó, Splay Tree cũng không tránh khỏi hằng số đáng kể. Có thể tạm xem việc dùng Splay Tree duy trì đường đi là thiết kế dành riêng cho Link-Cut Trees; nếu không thì có phần dùng quá mức.

Điều này gợi ý ta suy nghĩ xem có tồn tại một cấu trúc dữ liệu khác cho trường hợp hình dạng cây không thay đổi, dùng một cây cân bằng không phải trả chi phí xoay cao để duy trì đường đi, mà vẫn có độ phức tạp lý thuyết xấu nhất giống lời giải 1 và 3 hay không. Suy nghĩ theo hướng đó dẫn tới lời giải 4 ở đây.

Từ lời giải 3 và 4 có thể thấy, khi giải một số bài toán, tối ưu cục bộ chưa chắc đem lại tối ưu toàn cục. Ta không nên cố chấp theo đuổi tối ưu cục bộ, mà cần luôn giữ góc nhìn toàn cục, nắm được liên hệ giữa tổng thể và cục bộ, để từ đó đạt tối ưu trên tổng thể. Đây chính là mục đích chính của tác giả khi viết bài này.

A Lời cảm ơn

Bạn Yuan Xinhao của Trường Trung học Changjun, Hồ Nam, đã giúp tác giả rất nhiều trong quá trình hoàn thành báo cáo lời giải này. Xin gửi lời cảm ơn tới bạn ấy.

References

- [1] Daniel D. Sleator, Robert Endre Tarjan, *A data structure for dynamic trees*, Journal of Computer and System Sciences, vol. 26, no. 3, pp. 362–391, June 1983.
- [2] Daniel Dominic Sleator, Robert Endre Tarjan, *Self-adjusting binary trees*, Proceedings of the fifteenth annual ACM Symposium on Theory of Computing, pp. 235–245, December 1983.
- [3] Zhou Gelin, *Báo cáo lời giải “Query on a tree (SPOJ375)”*.